

SIMATS ENGINEERING

ASSESSMENT TOOL 2: Research Review & Analysis

COURSE NAME:	ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS	COURSE CODE:	MLA0101
--------------	---	--------------	---------

COURSE OUTCOME COVERED

CO4: *Develop intelligent software agents using suitable agent architectures and learning techniques.*(BTL 3)

Assessment Tool Weightage: 30%

Total Marks: 100

Time: 90 Mins

No. of Questions: 1

Annexure A

Research Review & Analysis

Code of Conduct:

*I **Yoga Madha A-192425058** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:



Criterion	Weightage	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Literature Search and Source Selection	10%	
3. Literature Review and Synthesis	15%	
4. Comparative Analysis of AI Techniques	10%	
5. Critical Analysis of Research Findings	10%	
6. Research Gap Identification	10%	
7. Proposed Research Direction	10%	
8. Research Methodology / Analysis Framework	5%	
9. Experimental / Evidence-Based Validation	10%	
10. Result Analysis and Interpretation	5%	
11. Research Paper Quality and Referencing	10%	
12. Technical Presentation and Research Defense	5%	
Total	100%	

Signature of the Faculty

Total Marks Awarded

ANSWER ANY ONE QUESTION

A RESEARCH REVIEW AND COMPARATIVE ANALYSIS OF Q-LEARNING, SARSA, AND DEEP Q-NETWORKS

Abstract

Reinforcement Learning (RL) is an important area of Artificial Intelligence in which an agent learns to make decisions by interacting with an environment and receiving rewards or penalties. Among the most influential value-based RL algorithms are Q-Learning, SARSA, and Deep Q-Networks (DQN). Q-Learning introduced an off-policy temporal-difference learning approach that estimates the optimal action-value function, while SARSA uses an on-policy strategy that learns from the actions actually selected by the agent. DQN extended Q-Learning by replacing the traditional Q-table with a deep neural network, allowing reinforcement learning to operate in high-dimensional state spaces such as images.

This review analyzes research on these three approaches based on learning mechanisms, exploration strategies, convergence, computational requirements, scalability, generalization, and real-world applications. The review shows that traditional Q-Learning and SARSA are computationally simple and effective for small discrete environments, but their Q-tables become impractical as the state and action spaces increase. DQN provides substantially better scalability and representation learning, but requires greater computational resources, careful training procedures, and mechanisms such as experience replay and target networks. Research on Double Q-Learning and Rainbow further demonstrates that reducing value overestimation and combining complementary improvements can enhance DQN performance.

A major research gap is the difficulty of achieving reliable, sample-efficient, safe, and computationally affordable reinforcement learning in complex real-world environments. Future research should focus on adaptive exploration, efficient experience utilization, safety constraints, uncertainty estimation, and lightweight deep RL models for real-time applications.

Keywords: Reinforcement Learning, Q-Learning, SARSA, Deep Q-Network, Temporal-Difference Learning, Deep Reinforcement Learning

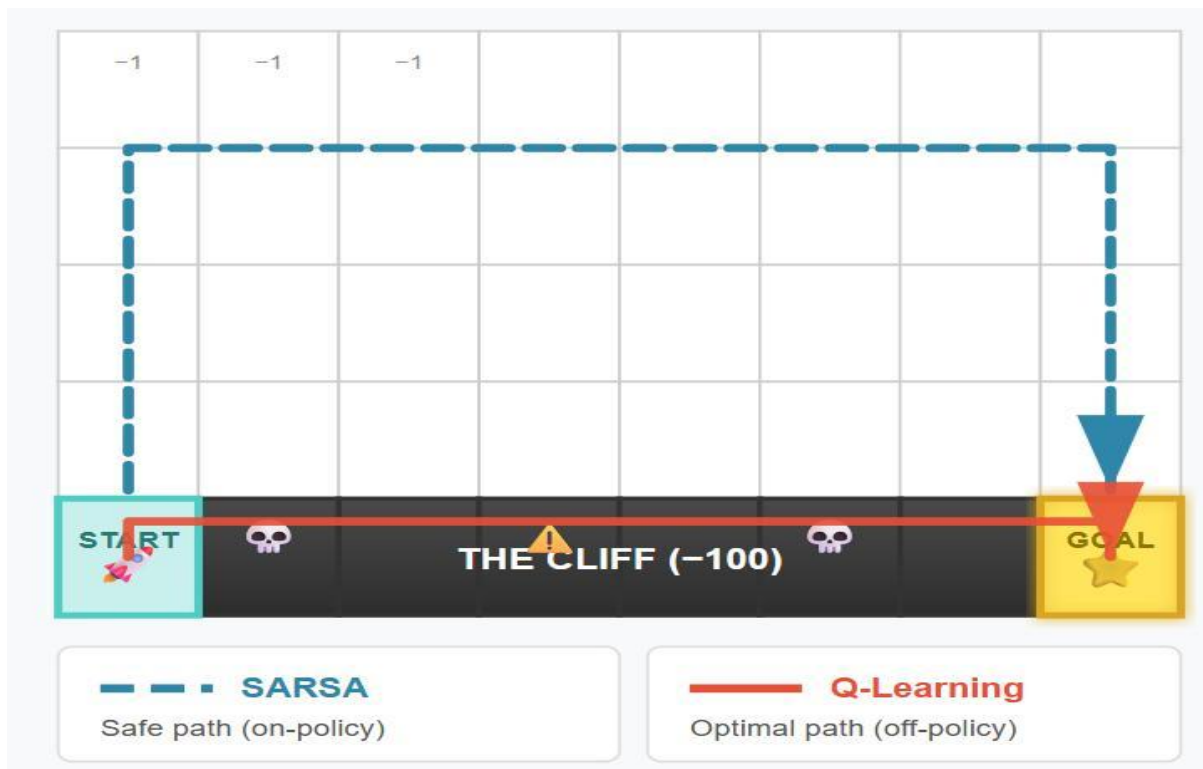
1. Introduction

Reinforcement Learning (RL) is a machine learning paradigm in which an intelligent agent learns an appropriate sequence of actions through interaction with an environment. Unlike supervised learning, where training data are explicitly labelled, reinforcement learning uses rewards and penalties to guide the learning process. The objective of the agent is to learn a policy that maximizes the cumulative reward over time.

Reinforcement learning has become important for applications involving sequential decision-making, including robotics, autonomous systems, games, resource management, traffic control, recommendation systems, and industrial automation.

A fundamental concept in RL is the **action-value function**, commonly represented as $Q(s,a)$, which estimates the expected future reward obtained by taking action a in state s . Q-Learning uses this concept to learn an optimal policy without requiring a model of the environment. Watkins and Dayan established the theoretical foundation of Q-Learning and showed convergence under suitable conditions, including repeated sampling of state-action pairs and discrete action-value representations.

SARSA is another temporal-difference learning method. Its name comes from the sequence:
State → Action → Reward → State → Action



Unlike Q-Learning, SARSA is an on-policy method because it updates its value estimates according to the action actually selected by the current policy.

Although Q-Learning and SARSA work effectively in small discrete environments, traditional Q-tables have serious limitations for large or continuous state spaces. The development of Deep Q-Networks addressed this limitation by using deep neural networks to approximate the Q-function. Mnih et al. demonstrated that DQN could learn directly from high-dimensional visual inputs and achieved strong performance across 49 Atari 2600 games.

Therefore, studying the evolution from Q-Learning and SARSA to DQN is important for understanding how reinforcement learning has progressed from simple tabular learning to deep reinforcement learning.

Scope of the Review

This review focuses mainly on:

1. Q-Learning
2. SARSA
3. Deep Q-Networks
4. Improvements to DQN such as Double Q-Learning and Rainbow
5. Learning mechanisms and exploration
6. Convergence and stability
7. Computational requirements
8. Scalability and generalization
9. Real-world applications
10. Current challenges and research gaps

2. Research Problem and Objectives

2.1 Research Problem

Traditional reinforcement learning algorithms such as Q-Learning and SARSA are effective in environments with relatively small and discrete state-action spaces. However, real-world applications often contain very large, continuous, noisy, and high-dimensional state spaces.

A Q-table becomes impractical when the number of possible states and actions increases significantly. For example, an autonomous vehicle cannot reasonably represent every possible combination of camera images, vehicle positions, traffic conditions, speeds, and environmental conditions using a conventional Q-table.

DQN attempts to solve this problem by using a neural network as a function approximator. However, deep reinforcement learning introduces new challenges, including high computational cost, instability, sample inefficiency, sensitivity to hyperparameters, and exploration difficulties.

Therefore, the central research problem is:

How do Q-Learning, SARSA, and DQN differ in learning mechanism, exploration, convergence, computational requirements, scalability, and practical applicability, and what improvements are required to make reinforcement learning more reliable for real-world environments

2.2 Objectives

The main objectives of this review are:

1. To review the fundamental principles of Q-Learning, SARSA, and DQN.
2. To analyze the learning mechanism of each algorithm.
3. To compare their exploration strategies.
4. To examine their convergence characteristics.
5. To compare computational requirements.
6. To evaluate their scalability and generalization capabilities.
7. To examine important applications of the algorithms.
8. To identify limitations and unresolved challenges.
9. To identify research gaps in current reinforcement learning research.
10. To propose possible future research directions.

3. Literature Review

3.1 Q-Learning

Q-Learning is one of the most influential model-free reinforcement learning algorithms. Watkins and Dayan formally presented Q-Learning in 1992. The method learns an action-value function that estimates the expected cumulative reward for each state-action pair.

The standard Q-Learning update equation is:

$$Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

where:

- $Q(s,a)$ = current estimate of action value
- α = learning rate
- r = immediate reward
- γ = discount factor

- s' = next state
- a' = possible next action

The important characteristic of Q-Learning is that it is **off-policy**. The agent may follow an exploratory behaviour policy while learning the value of the optimal greedy policy.

The theoretical work of Watkins and Dayan established convergence results for Q-Learning under appropriate assumptions.

Advantages of Q-Learning

- Simple algorithmic structure.
- Easy to implement.
- Does not require a model of the environment.
- Suitable for small discrete state-action spaces.
- Can learn an optimal policy under appropriate conditions.
- Off-policy learning allows flexible exploration.

Limitations of Q-Learning

- Q-tables become very large for complex environments.
- Poor scalability to high-dimensional state spaces.
- Requires sufficient exploration of state-action pairs.
- Traditional Q-Learning does not automatically learn useful representations.
- Can suffer from overestimation of action values.

Research on Double Q-Learning identified value overestimation as an important weakness of standard Q-Learning. Hasselt showed that the maximum operation used in Q-Learning can introduce positive bias and proposed Double Q-Learning to reduce this problem.

3.2 SARSA

SARSA is an on-policy temporal-difference reinforcement learning algorithm. The name represents the sequence of variables used in its update:

State $\rightarrow$ Action $\rightarrow$ Reward $\rightarrow$ State $\rightarrow$ Action

Its update equation can be represented as:

$$Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma Q(s',a') - Q(s,a)]$$

The main difference between SARSA and Q-Learning is the next-action value used in the update.

Q-Learning uses:

$$\max Q(s',a')$$

whereas SARSA uses:

$$Q(s',a')$$

for the action actually selected by the current policy.

Therefore, SARSA learns the value of the policy that the agent is actually following.

Characteristics of SARSA

SARSA is particularly useful when the consequences of exploratory actions need to be included in the learning process. Because it is on-policy, the learned behaviour can reflect the risk associated with the exploration strategy.

For example, if an agent is learning to navigate a dangerous environment, SARSA can learn a more conservative policy when exploration itself can lead to negative outcomes.

Advantages of SARSA

- Simple and computationally efficient.
- Suitable for small discrete environments.
- Incorporates the actual behaviour policy into learning.
- Can produce safer or more conservative behaviour in some environments.
- Easy to implement.

Limitations of SARSA

- Q-table scalability problem.
- Requires sufficient exploration.
- Not naturally suitable for high-dimensional sensory inputs.
- Performance depends on the behaviour policy.
- Less suitable for very large state spaces without function approximation.

Rummery and Niranjan investigated connectionist approaches to Q-learning because traditional finite-state approaches were too restrictive for many real-world environments. Their work considered neural-network-based function approximation for high-dimensional continuous state spaces, helping motivate later developments in function-approximation-based reinforcement learning.

3.3 Deep Q-Network

Deep Q-Networks represent a major development in reinforcement learning because they combine Q-Learning with deep neural networks.

Instead of storing a separate Q-value for every state-action pair, DQN uses a neural network to approximate:

$$Q(s,a; \theta)$$

where θ represents the neural-network parameters.

The network receives the state as input and predicts Q-values for possible actions.

A simplified DQN architecture is:

Environment $\rightarrow$ State $\rightarrow$ Deep Neural Network $\rightarrow$ Q-values $\rightarrow$ Action $\rightarrow$ Environment

Mnih et al. introduced a DQN system capable of learning from raw image pixels and demonstrated strong performance on Atari 2600 games. The system used the same algorithm, network architecture, and hyperparameters across 49 games, demonstrating the ability of deep RL to generalize across multiple tasks.

Important DQN Components

1. Deep Neural Network

The neural network approximates the Q-function and learns useful representations directly from the input state.

2. Experience Replay

Instead of training only on consecutive experiences, DQN stores previous transitions in a replay memory and samples mini-batches during training.

This helps reduce correlations between consecutive training samples and improves data utilization.

3. Target Network

DQN maintains a separate target network to calculate more stable target values.

This reduces the instability that can occur when the same network is simultaneously used to predict and generate target values.

4. ϵ -Greedy Exploration

DQN commonly uses ϵ -greedy exploration. With probability ϵ , the agent selects a random action; otherwise, it selects the action with the highest estimated Q-value.

Advantages of DQN

- Handles high-dimensional state representations.
- Learns features automatically.
- Avoids explicit Q-table storage.
- Can process visual information.
- Provides strong performance in complex environments.
- Can generalize across states.

Limitations of DQN

- High computational requirements.
- Requires large amounts of experience.
- Training can be unstable.
- Sensitive to hyperparameters.
- Exploration remains challenging.
- Primarily designed for discrete action spaces.
- Neural networks make the learned policy less interpretable.

3.4 Double Q-Learning and Double DQN

Standard Q-Learning can overestimate action values because it selects and evaluates the maximum action using the same estimator.

Hasselt proposed Double Q-Learning to address this overestimation problem. The method uses two value estimators, reducing the positive bias associated with the maximum operation.

The same idea was later extended to deep reinforcement learning.

Van Hasselt, Guez, and Silver demonstrated that DQN can suffer from substantial overestimation in some Atari games. They proposed a Double DQN approach that reduced overestimation and improved performance on several games.

This development demonstrates an important research pattern:

Basic algorithm → identified limitation → theoretical improvement → improved deep-learning implementation

3.5 Rainbow

Rainbow is another important development in deep reinforcement learning. Hessel et al. investigated the combination of several improvements to DQN.

The Rainbow approach combined six major improvements and evaluated whether they could work together effectively. Experiments on Atari 2600 showed that the combined method achieved strong performance in terms of data efficiency and final performance. The study also included an ablation analysis to determine the contribution of individual components.

Rainbow demonstrates that DQN performance can be improved by integrating multiple techniques rather than relying on a single modification.

3.6 Summary of Reviewed Research

Research Algorithm	Main Contribution	Major Strength	Major Limitation
Watkins & Dayan (1992) – Q-Learning	Established Q-Learning and convergence analysis	Simple and theoretically well founded	Poor scalability of Q-table
Rummery & Niranjan (1994) – Connectionist Q-Learning	Investigated function approximation	Extends RL toward larger state spaces	Neural approximation introduces complexity
Hasselt (2010) – Double Q-Learning	Reduced Q-value overestimation	Better value estimation	Requires additional estimator
Mnih et al. (2015) – DQN	Combined Q-Learning with deep neural networks	Handles high-dimensional inputs	High computational and data requirements
Van Hasselt et al. (2016) – Double DQN	Reduced DQN overestimation	Better stability/performance	Still computationally expensive
Hessel et al. (2018) – Rainbow	Combined multiple DQN improvements	Strong performance and data efficiency	Increased algorithmic complexity

4. Comparative Analysis

4.1 Learning Mechanism

Q-Learning

Q-Learning learns the optimal action-value function using the maximum estimated value of the next state.

It is:

Off-policy + Temporal Difference Learning

SARSA

SARSA updates the Q-value using the action actually selected by the current policy.

It is:

On-policy + Temporal Difference Learning

DQN

DQN uses a deep neural network to approximate the Q-function.

It is:

Deep Function Approximation + Q-Learning + Experience Replay + Target Network

4.2 Exploration Strategy

Q-Learning and DQN commonly use ϵ -greedy exploration. The agent occasionally selects a random action and otherwise chooses the action with the highest estimated value.

SARSA can also use ϵ -greedy exploration, but its learning update includes the action actually selected. Therefore, its learned policy is directly influenced by the exploration behaviour.

Algorithm	Common Exploration	Exploration Influence
Q-Learning	ϵ -greedy	Learns optimal greedy policy despite exploratory behaviour
SARSA	ϵ -greedy	Learns according to actual behaviour policy
DQN	ϵ -greedy	Exploration used with neural Q-function

4.3 Convergence

Q-Learning has strong theoretical convergence results under specific assumptions. Watkins and Dayan showed convergence to optimal action values under conditions including sufficient repeated exploration and discrete representations.

SARSA also has theoretical convergence properties under appropriate assumptions, but its learned values depend on the behaviour policy.

DQN is more difficult to analyze theoretically because neural networks are nonlinear function approximators and training uses correlated experience and bootstrapped targets.

Therefore:

Q-Learning → strongest simple theoretical foundation

SARSA → strong theoretical foundation under on-policy assumptions

DQN → more complex convergence and stability issues

4.4 Computational Requirements

Q-Learning has relatively low computational requirements when the state and action spaces are small.

SARSA has similar computational requirements.

DQN requires considerably more computational resources because it performs neural-network forward and backward propagation.

Parameter	Q-Learning	SARSA	DQN
Memory	Low–Medium	Low–Medium	High
CPU/GPU requirement	Low	Low	High
Training complexity	Low	Low	High
Q-table required	Yes	Yes	No
Neural network	No	No	Yes
Large state space	Poor	Poor	Good
Image input	Poor	Poor	Good

4.5 Scalability

Scalability is one of the biggest differences between traditional RL and deep RL.

For a small grid-world environment, Q-Learning and SARSA can efficiently store and update Q-values.

However, for large environments, the number of state-action pairs becomes extremely large.

DQN solves part of this problem by approximating the Q-function using a neural network.

Therefore:

Q-Learning and SARSA are suitable for small state spaces, whereas DQN is more suitable for high-dimensional state spaces.

Mnih et al.'s Atari experiments provided strong evidence that deep Q-learning can operate directly on high-dimensional sensory information.

4.6 Generalization

Traditional Q-Learning and SARSA store values for specific state-action pairs. Consequently, they have limited generalization.

For example, if an agent learns the value of:

State A + Action X

it does not automatically understand the value of a very similar but previously unseen state.

DQN uses neural networks that can learn representations and patterns from states. Therefore, DQN can generalize more effectively across similar states.

However, neural-network generalization can also introduce unexpected behaviour when the agent encounters states that differ significantly from the training distribution.

4.7 Interpretability

Q-Learning and SARSA are relatively easy to interpret because their Q-values can be inspected directly.

DQN is less interpretable because decisions are produced by a deep neural network containing many parameters.

Therefore:

Interpretability: Q-Learning $\approx$ SARSA > DQN

For safety-critical applications, this difference is important.

4.8 Overall Comparison

Criteria	Q-Learning	SARSA	DQN
Learning type	Off-policy	On-policy	Off-policy
Representation	Q-table	Q-table	Neural network
State space	Small/discrete	Small/discrete	Large/high-dimensional
Exploration	ϵ -greedy commonly used	ϵ -greedy commonly used	ϵ -greedy commonly used
Computational cost	Low	Low	High
Memory requirement	Low–Medium	Low–Medium	High
Generalization	Limited	Limited	High
Interpretability	High	High	Low
Training stability	Generally high in simple settings	Generally high in simple settings	More difficult

Criteria	Q-Learning	SARSA	DQN
Image processing	Not suitable directly	Not suitable directly	Suitable
Scalability	Low	Low	High
Implementation complexity	Low	Low	High
Suitable applications	Small control problems	Risk-sensitive/simple control	Games, robotics, complex environments

5. Critical Analysis

The reviewed research shows that there is no single reinforcement learning algorithm that is optimal for every application.

5.1 Strengths of Q-Learning

The main advantage of Q-Learning is its simplicity. It can be implemented using a relatively small amount of computation and does not require a model of the environment.

Its theoretical foundation is also strong. The convergence results provided by Watkins and Dayan established an important basis for subsequent reinforcement learning research.

However, the algorithm is fundamentally limited by the size of the Q-table.

5.2 Strengths of SARSA

SARSA provides a useful alternative because it learns from the actual behaviour of the agent. This can be beneficial in situations where exploratory actions have consequences. The on-policy nature of SARSA can result in more conservative behaviour compared with purely learning the optimal greedy policy.

However, SARSA still faces the state-space scalability problem when implemented using a conventional Q-table.

5.3 Strengths of DQN

DQN represents a major advance because it combines reinforcement learning with deep representation learning.

The Atari study demonstrated that DQN could learn from raw pixels and achieve strong performance over many different games.

This significantly expanded the range of environments that could be addressed using reinforcement learning.

5.4 Weaknesses of DQN

Despite its success, DQN requires substantial computational resources and large amounts of interaction data.

The agent may need millions of environment interactions before learning an effective policy. This is a major limitation for real-world applications where collecting experience may be expensive, slow, or dangerous.

DQN also has problems related to value overestimation. Research on Double DQN demonstrated that these overestimations can occur in practice and that reducing them can improve performance.

6. Research Gap

The literature demonstrates significant progress from tabular reinforcement learning to deep reinforcement learning. However, several important research gaps remain.

6.1 Sample Efficiency

DQN-based methods may require a large amount of interaction data. This is acceptable in simulated environments but difficult in physical systems such as robots, autonomous vehicles, and industrial machines.

Research gap: More sample-efficient learning methods are required.

6.2 Safe Exploration

Random exploration can be dangerous in real-world applications.

For example, an autonomous vehicle cannot randomly select unsafe driving actions simply to explore the environment.

Research gap: Exploration strategies need to consider safety constraints and risk.

6.3 Computational Cost

Deep reinforcement learning requires significant computing resources.

This creates challenges for deployment on edge devices, embedded systems, and low-power hardware.

Research gap: Lightweight reinforcement learning models are required for real-time applications.

6.4 Generalization

DQN can generalize better than a Q-table, but performance may degrade when the environment changes significantly.

For example, an agent trained in one traffic environment may not automatically perform well in a different city with different road layouts and traffic patterns.

Research gap: Robust transfer and generalization mechanisms are required.

6.5 Explainability

Traditional Q-Learning provides relatively understandable Q-values, whereas DQN decisions are harder to interpret.

In safety-critical systems, users may need to understand why an agent selected a particular action.

Research gap: Explainable deep reinforcement learning remains an important research area.

6.6 Stability and Reliability

Deep RL can be sensitive to hyperparameters, reward design, random seeds, and environment conditions.

Two training runs can sometimes produce different performance.

Research gap: More reliable, reproducible, and stable deep reinforcement learning methods are required.

7. Proposed Research Direction

Based on the identified gaps, a possible future research direction is the development of a:

Safe, Adaptive, and Lightweight Deep Reinforcement Learning Framework

The proposed framework can combine the strengths of traditional and deep RL methods.

Proposed Components

1. Deep Q-Network

Use DQN to handle high-dimensional state information.

2. Double DQN

Use Double Q-learning principles to reduce Q-value overestimation. Research has shown that Double DQN can reduce overestimation and improve performance compared with standard DQN in several environments.

3. Prioritized Experience Replay

Give greater training importance to experiences that contain more useful learning information.

4. Adaptive Exploration

Instead of using a fixed exploration schedule, dynamically modify exploration according to uncertainty and learning progress.

5. Safety Constraints

Prevent actions that violate predefined safety conditions.

6. Transfer Learning

Reuse knowledge learned in one environment when moving to a related environment.

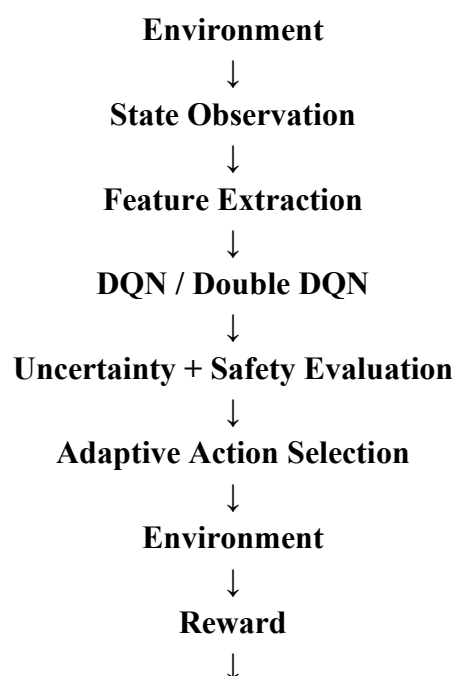
7. Explainability Module

Provide information about the major state features that influenced the selected action.

8. Lightweight Network

Reduce neural-network size to allow deployment on edge devices.

Proposed Architecture



Prioritized Experience Replay



Network Update



Improved Policy

This approach could provide a better balance between performance, safety, computational efficiency, and interpretability.

8. Discussion

The literature demonstrates a clear evolutionary path in reinforcement learning.

The first stage involves **tabular reinforcement learning**, represented by algorithms such as Q-Learning and SARSA. These methods are simple, computationally efficient, and relatively interpretable. Their major limitation is scalability.

The second stage involves **function approximation**, where neural networks are used to represent value functions. Early work on connectionist Q-learning explored the use of neural networks to extend reinforcement learning beyond finite discrete state spaces.

The third stage is represented by **deep reinforcement learning**, particularly DQN. DQN demonstrated that reinforcement learning could operate directly on high-dimensional sensory information and achieve strong performance on complex tasks.

The fourth stage involves **improved deep RL algorithms**, such as Double DQN and Rainbow. Double DQN addresses value overestimation, while Rainbow combines multiple improvements to obtain stronger performance and data efficiency.

This progression shows that reinforcement learning research is increasingly focused not only on maximizing reward but also on improving:

- Sample efficiency
- Stability
- Generalization
- Safety
- Computational efficiency
- Robustness
- Explainability

For real-world deployment, these factors are as important as raw reward performance.

9. Conclusion

This research review compared Q-Learning, SARSA, and Deep Q-Networks based on their learning mechanisms, exploration strategies, convergence, computational requirements, scalability, generalization, interpretability, and applications.

Q-Learning provides a simple and effective off-policy approach for learning optimal action values. Its major strengths are simplicity, low computational cost, and strong theoretical foundations. However, its Q-table limits scalability.

SARSA is an on-policy alternative that learns according to the behaviour policy actually followed by the agent. It is computationally efficient and useful for environments where the consequences of exploration need to be considered. However, it also suffers from the limitations of tabular representations.

DQN extends Q-Learning using deep neural networks and provides the ability to process high-dimensional state information. The work of Mnih et al. demonstrated the effectiveness of DQN in Atari games and became an important milestone in deep reinforcement learning research.

Further research has improved DQN through approaches such as Double Q-Learning and Rainbow. Double Q-Learning addresses action-value overestimation, while Rainbow demonstrates that multiple complementary improvements can be combined to improve deep RL performance.

The major research gap is that high-performing deep reinforcement learning systems can still be computationally expensive, data-intensive, difficult to explain, and challenging to deploy safely in real-world environments.

Therefore, future research should focus on **sample-efficient, safe, adaptive, explainable, and lightweight deep reinforcement learning systems**. Such developments can help bridge the gap between successful reinforcement learning experiments in simulation and reliable deployment in real-world applications such as autonomous vehicles, robotics, smart traffic management, industrial automation, and intelligent control systems.

References

- Hasselt, H. V. (2010). Double Q-learning. In *Advances in Neural Information Processing Systems 23* (pp. 2613–2621). Neural Information Processing Systems.
- Hessel, M., Modayil, J., van Hasselt, H., Schaul, T., Ostrovski, G., Dabney, W., Horgan, D., Piot, B., Azar, M., & Silver, D. (2018). Rainbow: Combining improvements in deep reinforcement learning. *Proceedings of the AAAI Conference on Artificial Intelligence*, 32(1).
- Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S., & Hassabis, D. (2015). Human-level control through deep reinforcement learning. *Nature*, 518, 529–533. <https://doi.org/10.1038/nature14236>.
- Rummery, G. A., & Niranjan, M. (1994). *On-line Q-learning using connectionist systems* (Technical Report CUED/F-INFENG/TR 166). Cambridge University Engineering Department.
- van Hasselt, H., Guez, A., & Silver, D. (2016). Deep reinforcement learning with Double Q-learning. *Proceedings of the AAAI Conference on Artificial Intelligence*, 30(1). <https://doi.org/10.1609/aaai.v30i1.10295>.
- Watkins, C. J. C. H., & Dayan, P. (1992). Q-learning. *Machine Learning*, 8, 279–292. <https://doi.org/10.1007/BF00992698>.